

Manual de Engenharia & Arquitetura de Software — Ecossistema NOVA

Autor: Fábio Rodrigues (Recife/PE)

Stack Core: Java 21 LTS, Spring Boot 3.3.3, Clean Architecture, Spring AI MCP, H2 Database (ACID), Python, Voice AI, Material 3 Expressive, GitHub Actions.

Repositório Oficial: <https://github.com/fabiorodrigues-tech-dev/NOVA>

Perfil Profissional: <https://linkedin.com/in/fabiorodrigues-dev>

Sumário Executivo

O **NOVA** é um ecossistema multi-agente pessoal e profissional orientado a microsserviços e inteligência artificial autônoma. Desenvolvido sob rigorosos princípios de **Clean Architecture**, **SOLID** e **DevSecOps**, o sistema atua como copiloto de engenharia de software, esteira automatizada de carreiras 360°, inteligência preditiva financeira e síntese de voz neural em alta fidelidade.

Seção 1 — Arquitetura de Microsserviços & Clean Architecture

A arquitetura do microsserviço principal (**agente-financeiro**) foi estruturada em camadas concêntricas e estritamente desacopladas, assegurando que as regras de negócio centrais sejam totalmente agnósticas a frameworks, bancos de dados ou interfaces de entrega.

1.1. Estrutura de Camadas (Ports & Adapters)

```
com.nova.agente financeiro /
├── domain /                # [CAMADA NÚCLEO]
│   ├── model /            # Entidades puras (Transacao, Caixinha, ResumoFinanceiro)
│   └── repository /       # Portas de Saída (Interfaces TransacaoRepository, CaixinhaRepository)
├── application /          # [CAMADA DE CASOS DE USO]
│   ├── dto /              # Records imutáveis de entrada/saída (TransacaoRequest, ProjecaoResponse)
│   └── usecase /          # Lógica de negócio (Cadastrar, Listar, Projeção, OFX Parser, Caixinhas)
├── infrastructure /       # [CAMADA DE ADAPTADORES & FRAMEWORKS]
│   ├── persistence /     # Adaptadores JPA, Entidades de Banco, Mappers bidirecionais
│   ├── web /              # Controllers REST (RFC 7807) e Exception Handler Global
│   ├── mcp /              # Tools corporativas Spring AI Model Context Protocol (@Tool)
│   └── config /           # Configurações de Beans e contexto Spring
```

1.2. Princípios SOLID Aplicados

- Single Responsibility Principle (SRP):** Cada Use Case possui uma única razão para mudar (ex: **CalcularProjecaoFinanceiraUseCase** foca estritamente no algoritmo preditivo).
- Open/Closed Principle (OCP):** Novos formatos de importação ou novos adaptadores de persistência são adicionados via implementação de interfaces sem alterar as regras de domínio existentes.
- Liskov Substitution Principle (LSP):** As implementações JPA em **infrastructure.persistence** respeitam integralmente os contratos definidos nas interfaces do domínio.
- Interface Segregation Principle (ISP):** Interfaces granulares e focadas, evitando que adaptadores dependam de métodos que não utilizam.
- Dependency Inversion Principle (DIP):** O domínio não conhece o Spring nem o JPA. A injeção de dependência ocorre das camadas externas para as internas através de construtores com tipagem forte.

1.3. Tratamento Global de Erros (RFC 7807 ProblemDetail)

O sistema implementa tratamento unificado de exceções via **@RestControllerAdvice** e **GlobalExceptionHandler**, emitindo payloads padronizados com timestamp, status HTTP semântico, detalhamento do erro e URI canônica para observabilidade corporativa.

Seção 2 — Protocolo MCP & Integração de IA (Spring AI)

O **Model Context Protocol (MCP)** é o padrão de interoperabilidade que conecta o modelo de linguagem aos sistemas corporativos locais de forma determinística e segura.

2.1. Ferramentas Corporativas (@Tool)

Através do Spring AI MCP Server, métodos Java são expostos como ferramentas corporativas acionáveis por LLMs (Gemini, Claude, GPT):

- **consultar_resumo_financeiro**: Retorna KPIs orçamentários, balanço consolidado e distribuição por categoria.
- **consultar_projecao_financeira**: Executa a projeção preditiva de fechamento com cálculo de Burn Rate.
- **atualizar_caixinha & consultar_caixinhas**: Gestão de alocação de reservas e metas com recálculo de Patrimônio Líquido Total.
- **processar_notificacao_nubank**: Webhook semântico para conciliação automática de pagamentos e transferências.

2.2. Agente Autônomo de Voz — Sofia (Infinitt Tecnologia)

No ecossistema paralelo de prospecção e atendimento B2B, a assistente **Sofia** opera conectada via Vapi, orquestrando fluxos com:

- **Speech-to-Text (STT)**: Transcrição de áudio em tempo real com alta acurácia em português brasileiro.
- **LLM Reasoning**: Processamento semântico com inferência rápida para qualificação comercial de equipamentos gráficos.
- **Text-to-Speech (TTS)**: Síntese neural natural e humanizada.
- **Webhooks Assíncronos & Mensageria**: Registro de dados em back-end RESTful e disparo automático de confirmações comerciais.

Seção 3 — Qualidade de Software, Testes & CI/CD

3.1. Pirâmide de Testes Automatizados (JUnit 5 + Mockito)

A suíte conta com **40 testes automatizados** cobrindo 100% dos fluxos críticos de negócio:

- **Testes Unitários**: Validação isolada de todos os 8 Casos de Uso, parsers e regras de domínio sem depender de Spring context.
- **Testes de Integração WebMvc**: Simulação de requisições HTTP REST (**MockMvc**) com validação de contratos JSON, status codes e payloads RFC 7807.
- **Testes de Ferramentas MCP**: Validação das chamadas **@Tool** para garantir respostas determinísticas para o agente inteligente.

3.2. Pipeline de CI/CD no GitHub Actions (.github/workflows/ci.yml)

Esteira automatizada executando em containers Ubuntu a cada push na branch **main**:

- **Job 1 (Java 21 & Maven)**: Setup do JDK 21 Temurin, compilação de classes e execução da suíte completa de testes com publicação de relatórios Surefire.
- **Job 2 (Python Quality & Voice Check)**: Verificação de sintaxe estática (Flake8), compilação de scripts e validação de compatibilidade *headless*.

3.3. Persistência ACID & Parser OFX

- **Banco H2 Local**: Persistência transacional com banco de dados H2 gravado em arquivo local (`./data/financiadb.mv.db`), garantindo isolamento ACID completo.
- **Parser OFX Nativo**: Varredura automática do diretório **financeiro/extratos_ofx/**, extração de tags SGML/XML (, ,), categorização inteligente e deduplicação rigorosa contra duplicidade no banco.

Seção 4 — Inteligência Preditiva & Consultoria Financeira (Fase 9)

O módulo preditivo atua como um CFO algorítmico automatizado:

- **Burn Rate Diário**: Calcula a média ponderada de gastos por dia decorrido no ciclo mensal:

$$\text{Burn Rate} = \frac{\text{Total de Gastos Acumulados}}{\text{Dias Decorridos}}$$

- **Gasto Adicional & Total Projetado**:

$$\text{Gasto Total Projetado} = \text{Gastos Atuais} + (\text{Burn Rate} \times \text{Dias Restantes})$$

- **Saldo Final Projetado**:

$$\text{Saldo Final} = \text{Receitas Atuais} - \text{Gasto Total Projetado}$$

- **Classificação Orçamentária & Recomendações**: Classifica o estado em **SAUDÁVEL**, **ALERTA** ou **CRÍTICO**, emitindo diagnósticos automáticos e sugestões de alocação estratégica em caixinhas.

Seção 5 — Esteira de Carreira 360° & Agentes Especialistas

O ecossistema opera através de 4 agentes especializados orquestrados pelo MAIN Agent:

1. **Agente Código (agente-codigo):** Clean Architecture, Java 21, Spring Boot, Scaffolding e testes automatizados.
 2. **Agente Estudos (agente-estudos):** Trilha Santander 2026 DIO, Técnica Feynman, Flashcards e desafios práticos JUnit 5.
 3. **Agente Carreira & Operações (agente-carreira-e-operacoes):** Gestão 360° de candidaturas segmentadas em duas trilhas oficiais:
 - **Trilha Tech & Dev:** Currículos Harvard Tech, LinkedIn oficial e cartas timbradas.
 - **Trilha Marketing & Audiovisual:** Portfólio Google Drive, cases reais e setup Apple Silicon M1 (Final Cut, DaVinci, Logic Pro).
 4. **Agente Financeiro (agente-financeiro):** Conciliação bancária, métricas orçamentárias e relatórios executivos.
-

Seção 6 — Links Oficiais & Acessos

- **Repositório Oficial no GitHub:** <https://github.com/fabiorodrigues-tech-dev/NOVA>
- **Perfil do Desenvolvedor no LinkedIn:** <https://linkedin.com/in/fabiorodrigues-dev>
- **Dossiê Técnico Master (PDF):** [docs/dossie_tecnico_nova.pdf](#)
- **NOVA Control Center (Local):** <http://localhost:3000>